

Ecommerce Project Documentation

Architecture, Pages, Backend, Database, APIs, Security, and Business Flows

Current implementation study guide

Generated from the source code in D:\projects\Ecommerce

Technology: Node.js, Express 5, MySQL, Vanilla JavaScript, HTML, CSS

Document date: 30 August 2026

How to Use This Document

This document explains the code that currently exists. It separates frontend behavior, backend routing, authorization, business logic, and database effects. It does not describe imagined future functionality.

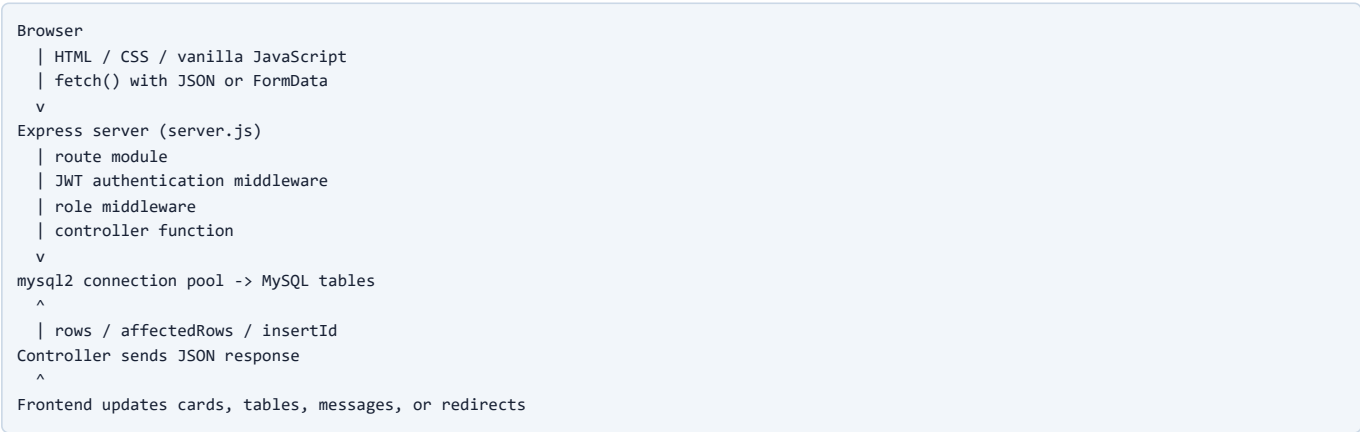
Actual implementation note. The admin area currently provides product approval, protected admin registration, customer lookup for Reward Points, and Reward Point crediting. It does not implement general customer deletion, seller administration, or category/brand CRUD. Customers also have no customer-side order-cancellation endpoint; cancellation is performed through the seller order workflow for eligible single-seller orders.

Contents

Part	Subject
1	Project overview and architecture
2	Complete folder structure
3	All 22 frontend pages
4	Every backend JavaScript file
5	Authentication
6	Multiple-role system
7	All 18 database tables
8	Major business flows
9	Reward Point system
10	Checkout and transactions
11	Complete API reference
12	Important code explained
13	Security review
14	Error handling
15	What happens when I click...
16	File-by-file quick reference
17	Final system map

Part 1 - Project Overview

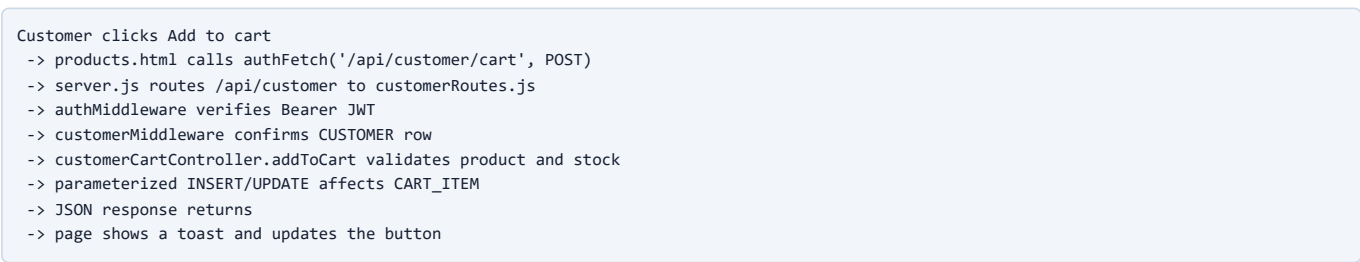
This is a classic three-layer web application. The browser serves the user interface, Express exposes HTTP APIs and executes business rules, and MySQL stores persistent data.



Layers and technologies

Technology/layer	Use in this project
Node.js	Runs the CommonJS backend JavaScript and starts the HTTP server.
Express 5	Serves public files, parses JSON, mounts four route groups, and dispatches requests.
MySQL	Stores users, roles, products, orders, wallets, reviews, and related records.
mysql2/promise	Provides parameterized promise-based queries, pooled connections, and transactions.
JWT / jsonwebtoken	Creates a signed one-day token containing only userId; verifies protected requests.
bcrypt	Hashes new passwords and verifies submitted passwords without storing plaintext.
dotenv	Loads configuration such as DB_HOST, DB_USER, DB_PASSWORD, DB_NAME, and JWT_SECRET.
Multer	Accepts seller product-image uploads, validates file types/count/size, and writes under public/uploads/products.
Vanilla frontend	Each HTML page owns its small page script; panel-common.js supplies shared authenticated fetch and UI helpers.
seller.css	One shared visual language for customer, seller, admin, authentication, cards, forms, tables, and sidebars.

Request lifecycle example



Part 2 - Complete Folder Structure

Ecommerce/
.git/FETCH_HEAD
.git/HEAD
.git/config
.git/description
.git/hooks/applypatch-msg.sample
.git/hooks/commit-msg.sample
.git/hooks/fsmonitor-watchman.sample
.git/hooks/post-update.sample
.git/hooks/pre-applypatch.sample
.git/hooks/pre-commit.sample
.git/hooks/pre-merge-commit.sample
.git/hooks/pre-push.sample
.git/hooks/pre-rebase.sample
.git/hooks/pre-receive.sample
.git/hooks/prepare-commit-msg.sample
.git/hooks/push-to-checkout.sample
.git/hooks/sendemail-validate.sample
.git/hooks/update.sample
.git/index
.git/info/exclude
.git/logs/HEAD
.git/logs/refs/heads/main
.git/logs/refs/remotes/origin/HEAD
.git/objects/pack/pack-9e57f9fed8fc87ebc57fe983805785f3e41d8cb9.idx
.git/objects/pack/pack-9e57f9fed8fc87ebc57fe983805785f3e41d8cb9.pack
.git/objects/pack/pack-9e57f9fed8fc87ebc57fe983805785f3e41d8cb9.rev
.git/packed-refs
.git/refs/heads/main
.git/refs/remotes/origin/HEAD
.gitignore
controllers/adminController.js
controllers/authController.js
controllers/customerAccountController.js
controllers/customerAddressController.js
controllers/customerCartController.js
controllers/customerCatalogController.js
controllers/customerController.js
controllers/customerOrderController.js
controllers/customerWalletController.js
controllers/customerWishlistController.js
controllers/sellerController.js
controllers/sellerDashboardController.js
controllers/sellerOrderController.js
controllers/sellerProductController.js
db/connection.js
db/ecommerce_dump.sql
middleware/adminMiddleware.js
middleware/authMiddleware.js
middleware/customerMiddleware.js
middleware/sellerMiddleware.js
middleware/uploadMiddleware.js
mydb.sql
package-lock.json
package.json
pass
public/addresses.html
public/admin.html
public/cart.html
public/checkout.html
public/customer.html
public/login.html
public/order-details.html
public/orders.html
public/panel-common.js
public/product-details.html
public/products.html
public/register_admin.html
public/register_customer.html
public/register_seller.html
public/role-selection.html
public/search-history.html
public/seller-orders.html
public/seller-product-form.html
public/seller-product-view.html
public/seller-products.html
public/seller.css
public/seller.html

```
public/wallet.html
public/wishlist.html
routes/adminRoutes.js
routes/authRoutes.js
routes/customerRoutes.js
routes/sellerRoutes.js
scripts/seedData.js
server.js
```

Folder responsibilities

Folder/file	Responsibility
controllers/	SQL-backed request handlers and business logic. Feature controllers are combined by small <code>customerController</code> and <code>sellerController</code> facades.
routes/	Maps HTTP method/path combinations to middleware and controller functions.
middleware/	Verifies JWTs, role membership, admin activity, and uploaded images.
public/	Twenty-two browser pages, shared CSS, shared panel JavaScript, and runtime uploads.
db/connection.js	Exports the mysql2 promise pool configured from environment values.
db/ecommerce_dump.sql	Authoritative simplified 18-table schema.
scripts/seedData.js	Destructive development seed script that clears and inserts representative data.
server.js	Creates Express, installs JSON/static middleware, mounts routes, and listens on PORT.
package.json / package-lock.json	Dependency declarations and exact dependency lock.
mydb.sql	Legacy SQL file; it is not the authoritative current schema.
.env	Local secrets/configuration; intentionally excluded from this document.

Part 3 - Every Frontend Page

Each protected panel page loads panel-common.js. authFetch attaches Authorization: Bearer <token>, converts ordinary object bodies to JSON, preserves FormData for uploads, and redirects to login after HTTP 401. Role-specific middleware remains the real authorization boundary.

Frontend Page: addresses.html

A. Purpose

CRUD and default selection for customer-owned addresses.

B. Access

Customer

C. UI and design

Uses the shared seller.css design: slate page background, dark collapsible role sidebar where appropriate, white panels/cards, blue primary buttons, compact fields, tables, tags, toasts, loading and empty states. Authentication pages use the shared centered auth-card variation.

D. Important JavaScript

- escapeHtml
- drawAddresses
- loadAddresses
- clearForm
- editAddress
- setDefault
- deleteAddress
- load/draw addresses
- create or update shared form
- setDefault
- deleteAddress

E. Backend communication

- /api/customer/addresses
- /api/customer/addresses/

```
addresses.html
-> browser event / initial load
-> fetch or authFetch
-> Express mounted route
-> authentication + role middleware when protected
-> controller SQL/business logic
-> JSON response
-> DOM update, toast, or redirect
```

F. User flow

The page validates immediately useful browser input, disables relevant buttons during asynchronous work, then relies on backend validation for ownership, stock, roles, balances, and database integrity. It handles non-success responses by showing the controller's message instead of assuming success.

Frontend Page: admin.html

A. Purpose

Product approval plus customer Reward Point crediting.

B. Access

Admin

C. UI and design

Uses the shared seller.css design: slate page background, dark collapsible role sidebar where appropriate, white panels/cards, blue primary buttons, compact fields, tables, tags, toasts, loading and empty states. Authentication pages use the shared centered auth-card variation.

D. Important JavaScript

- loadProducts
- setStatus
- loadRewardCustomers
- showCurrentPoints
- loadProducts/setStatus
- loadRewardCustomers
- POST Reward Point credit
- auto-refresh products

E. Backend communication

- /api/admin/customers
- /api/admin/customers/
- /api/admin/products/
- /api/admin/products?status=

```
admin.html
-> browser event / initial load
-> fetch or authFetch
-> Express mounted route
-> authentication + role middleware when protected
-> controller SQL/business logic
-> JSON response
-> DOM update, toast, or redirect
```

F. User flow

The page validates immediately useful browser input, disables relevant buttons during asynchronous work, then relies on backend validation for ownership, stock, roles, balances, and database integrity. It handles non-success responses by showing the controller's message instead of assuming success.

Frontend Page: cart.html

A. Purpose

Displays cart totals and allows quantity changes/removal before checkout.

B. Access

Customer

C. UI and design

Uses the shared seller.css design: slate page background, dark collapsible role sidebar where appropriate, white panels/cards, blue primary buttons, compact fields, tables, tags, toasts, loading and empty states. Authentication pages use the shared centered auth-card variation.

D. Important JavaScript

- `escapeHtml`
- `unavailableMessage`
- `drawCart`
- `loadCart`
- `changeQuantity`
- `removeItem`
- `loadCart/drawCart`
- PATCH quantity
- DELETE item
- disable checkout for unavailable stock

E. Backend communication

- `/api/customer/cart`
- `/api/customer/cart/`

```
cart.html
-> browser event / initial load
-> fetch or authFetch
-> Express mounted route
-> authentication + role middleware when protected
-> controller SQL/business logic
-> JSON response
-> DOM update, toast, or redirect
```

F. User flow

The page validates immediately useful browser input, disables relevant buttons during asynchronous work, then relies on backend validation for ownership, stock, roles, balances, and database integrity. It handles non-success responses by showing the controller's message instead of assuming success.

Frontend Page: checkout.html

A. Purpose

Loads cart/address/payment choices and atomically places an order, including full Reward Point payment.

B. Access

Customer

C. UI and design

Uses the shared seller.css design: slate page background, dark collapsible role sidebar where appropriate, white panels/cards, blue primary buttons, compact fields, tables, tags, toasts, loading and empty states. Authentication pages use the shared centered auth-card variation.

D. Important JavaScript

- escapeHtml
- drawCheckout
- loadCheckout
- toggleAddressForm
- addAddress
- placeOrder
- load three APIs in parallel
- draw payment and Reward Point eligibility
- add address
- POST checkout
- show confirmation and redirect

E. Backend communication

- /api/customer/addresses
- /api/customer/cart
- /api/customer/checkout
- /api/customer/checkout/options

```
checkout.html
-> browser event / initial load
-> fetch or authFetch
-> Express mounted route
-> authentication + role middleware when protected
-> controller SQL/business logic
-> JSON response
-> DOM update, toast, or redirect
```

F. User flow

The page validates immediately useful browser input, disables relevant buttons during asynchronous work, then relies on backend validation for ownership, stock, roles, balances, and database integrity. It handles non-success responses by showing the controller's message instead of assuming success.

Frontend Page: customer.html

A. Purpose

Displays the customer account profile in the shared customer panel layout.

B. Access

Customer

C. UI and design

Uses the shared seller.css design: slate page background, dark collapsible role sidebar where appropriate, white panels/cards, blue primary buttons, compact fields, tables, tags, toasts, loading and empty states. Authentication pages use the shared centered auth-card variation.

D. Important JavaScript

- escapeHtml
- loadCustomer
- setupSidebar/loadCustomerBox
- loadCustomer calls GET /api/customer/me
- escape and render identity fields

E. Backend communication

- /api/customer/me

```
customer.html
-> browser event / initial load
-> fetch or authFetch
-> Express mounted route
-> authentication + role middleware when protected
-> controller SQL/business logic
-> JSON response
-> DOM update, toast, or redirect
```

F. User flow

The page validates immediately useful browser input, disables relevant buttons during asynchronous work, then relies on backend validation for ownership, stock, roles, balances, and database integrity. It handles non-success responses by showing the controller's message instead of assuming success.

Frontend Page: login.html

A. Purpose

Authenticates an account with email/password; stores JWT and follows the server-selected redirect.

B. Access

Public

C. UI and design

Uses the shared seller.css design: slate page background, dark collapsible role sidebar where appropriate, white panels/cards, blue primary buttons, compact fields, tables, tags, toasts, loading and empty states. Authentication pages use the shared centered auth-card variation.

D. Important JavaScript

The page uses an inline form event handler rather than named functions.

- submit loginForm
- POST /api/auth/login
- display server error or save token and redirect

E. Backend communication

- /api/auth/login

```
login.html
-> browser event / initial load
-> fetch or authFetch
-> Express mounted route
-> authentication + role middleware when protected
-> controller SQL/business logic
-> JSON response
-> DOM update, toast, or redirect
```

F. User flow

The page validates immediately useful browser input, disables relevant buttons during asynchronous work, then relies on backend validation for ownership, stock, roles, balances, and database integrity. It handles non-success responses by showing the controller's message instead of assuming success.

Frontend Page: order-details.html

A. Purpose

Owned order header, address, products, totals, payment method, and transaction history.

B. Access

Customer

C. UI and design

Uses the shared seller.css design: slate page background, dark collapsible role sidebar where appropriate, white panels/cards, blue primary buttons, compact fields, tables, tags, toasts, loading and empty states. Authentication pages use the shared centered auth-card variation.

D. Important JavaScript

- fullDate
- escapeHtml
- addressText
- drawOrder
- loadOrder
- validate id query parameter
- GET owned order
- render items and Reward/Wallet transactions

E. Backend communication

- /api/customer/orders/

```
order-details.html
-> browser event / initial load
-> fetch or authFetch
-> Express mounted route
-> authentication + role middleware when protected
-> controller SQL/business logic
-> JSON response
-> DOM update, toast, or redirect
```

F. User flow

The page validates immediately useful browser input, disables relevant buttons during asynchronous work, then relies on backend validation for ownership, stock, roles, balances, and database integrity. It handles non-success responses by showing the controller's message instead of assuming success.

Frontend Page: orders.html

A. Purpose

Filtered and sorted order history.

B. Access

Customer

C. UI and design

Uses the shared seller.css design: slate page background, dark collapsible role sidebar where appropriate, white panels/cards, blue primary buttons, compact fields, tables, tags, toasts, loading and empty states. Authentication pages use the shared centered auth-card variation.

D. Important JavaScript

- loadOrders
- orderDate
- escapeHtml
- read status/sort query parameters
- GET /api/customer/orders
- render status/payment and detail links

E. Backend communication

- /api/customer/orders?status=

```
orders.html
-> browser event / initial load
-> fetch or authFetch
-> Express mounted route
-> authentication + role middleware when protected
-> controller SQL/business logic
-> JSON response
-> DOM update, toast, or redirect
```

F. User flow

The page validates immediately useful browser input, disables relevant buttons during asynchronous work, then relies on backend validation for ownership, stock, roles, balances, and database integrity. It handles non-success responses by showing the controller's message instead of assuming success.

Frontend Page: product-details.html

A. Purpose

Product gallery, specifications, seller details, reviews, cart and wishlist actions.

B. Access

Customer

C. UI and design

Uses the shared seller.css design: slate page background, dark collapsible role sidebar where appropriate, white panels/cards, blue primary buttons, compact fields, tables, tags, toasts, loading and empty states. Authentication pages use the shared centered auth-card variation.

D. Important JavaScript

- escapeHtml
- stars
- showImage
- step
- goTo
- drawProduct
- drawReviews
- loadProduct
- addToCart
- toggleWishlist
- submitReview
- markHelpful
- loadProduct by id
- gallery showImage/step/goTo
- addToCart/toggleWishlist
- submitReview and markHelpful

E. Backend communication

- /api/customer/cart
- /api/customer/products/
- /api/customer/reviews/
- /api/customer/wishlist/

```
product-details.html
-> browser event / initial load
-> fetch or authFetch
-> Express mounted route
-> authentication + role middleware when protected
-> controller SQL/business logic
-> JSON response
-> DOM update, toast, or redirect
```

F. User flow

The page validates immediately useful browser input, disables relevant buttons during asynchronous work, then relies on backend validation for ownership, stock, roles, balances, and database integrity. It handles non-success responses by showing the controller's message instead of assuming success.

Frontend Page: products.html

A. Purpose

Searchable/filterable/sortable storefront for active products.

B. Access

Customer

C. UI and design

Uses the shared seller.css design: slate page background, dark collapsible role sidebar where appropriate, white panels/cards, blue primary buttons, compact fields, tables, tags, toasts, loading and empty states. Authentication pages use the shared centered auth-card variation.

D. Important JavaScript

- escapeHtml
- stars
- drawFilters
- drawProducts
- buildQuery
- loadProducts
- addToCart
- toggleWishlist
- drawFilters/drawProducts
- buildQuery
- GET /api/customer/products
- POST cart and PATCH wishlist operations

E. Backend communication

- /api/customer/cart
- /api/customer/products
- /api/customer/wishlist/

```
products.html
-> browser event / initial load
-> fetch or authFetch
-> Express mounted route
-> authentication + role middleware when protected
-> controller SQL/business logic
-> JSON response
-> DOM update, toast, or redirect
```

F. User flow

The page validates immediately useful browser input, disables relevant buttons during asynchronous work, then relies on backend validation for ownership, stock, roles, balances, and database integrity. It handles non-success responses by showing the controller's message instead of assuming success.

Frontend Page: register_admin.html

A. Purpose

Allows an authenticated admin to assign ADMIN to an existing USER or create USER+ADMIN.

B. Access

Admin

C. UI and design

Uses the shared seller.css design: slate page background, dark collapsible role sidebar where appropriate, white panels/cards, blue primary buttons, compact fields, tables, tags, toasts, loading and empty states. Authentication pages use the shared centered auth-card variation.

D. Important JavaScript

The page uses an inline form event handler rather than named functions.

- loadAdminBox proves admin access
- POST /api/admin/register
- toast result and reset form

E. Backend communication

- /api/admin/register

```
register_admin.html
-> browser event / initial load
-> fetch or authFetch
-> Express mounted route
-> authentication + role middleware when protected
-> controller SQL/business logic
-> JSON response
-> DOM update, toast, or redirect
```

F. User flow

The page validates immediately useful browser input, disables relevant buttons during asynchronous work, then relies on backend validation for ownership, stock, roles, balances, and database integrity. It handles non-success responses by showing the controller's message instead of assuming success.

Frontend Page: register_customer.html

A. Purpose

Creates a new USER+CUSTOMER or adds CUSTOMER to an existing password-verified USER.

B. Access

Public

C. UI and design

Uses the shared seller.css design: slate page background, dark collapsible role sidebar where appropriate, white panels/cards, blue primary buttons, compact fields, tables, tags, toasts, loading and empty states. Authentication pages use the shared centered auth-card variation.

D. Important JavaScript

The page uses an inline form event handler rather than named functions.

- collect unchanged registration payload
- POST /api/auth/register_customer
- save returned token and open customer account

E. Backend communication

- /api/auth/register_customer

```
register_customer.html
-> browser event / initial load
-> fetch or authFetch
-> Express mounted route
-> authentication + role middleware when protected
-> controller SQL/business logic
-> JSON response
-> DOM update, toast, or redirect
```

F. User flow

The page validates immediately useful browser input, disables relevant buttons during asynchronous work, then relies on backend validation for ownership, stock, roles, balances, and database integrity. It handles non-success responses by showing the controller's message instead of assuming success.

Frontend Page: register_seller.html

A. Purpose

Creates USER+SELLER or adds SELLER to an existing password-verified USER.

B. Access

Public

C. UI and design

Uses the shared seller.css design: slate page background, dark collapsible role sidebar where appropriate, white panels/cards, blue primary buttons, compact fields, tables, tags, toasts, loading and empty states. Authentication pages use the shared centered auth-card variation.

D. Important JavaScript

The page uses an inline form event handler rather than named functions.

- collect identity and store fields
- POST /api/auth/register_seller
- save token and open seller dashboard

E. Backend communication

- /api/auth/register_seller

```
register_seller.html
-> browser event / initial load
-> fetch or authFetch
-> Express mounted route
-> authentication + role middleware when protected
-> controller SQL/business logic
-> JSON response
-> DOM update, toast, or redirect
```

F. User flow

The page validates immediately useful browser input, disables relevant buttons during asynchronous work, then relies on backend validation for ownership, stock, roles, balances, and database integrity. It handles non-success responses by showing the controller's message instead of assuming success.

Frontend Page: role-selection.html

A. Purpose

Lets a multi-role account choose only a role actually assigned by the database.

B. Access

Authenticated

C. UI and design

Uses the shared seller.css design: slate page background, dark collapsible role sidebar where appropriate, white panels/cards, blue primary buttons, compact fields, tables, tags, toasts, loading and empty states. Authentication pages use the shared centered auth-card variation.

D. Important JavaScript

- loadRoles
- chooseRole
- loadRoles calls GET /api/auth/roles
- render buttons from returned roles
- chooseRole calls POST /api/auth/select-role
- backend verifies membership and returns redirect

E. Backend communication

- /api/auth/roles
- /api/auth/select-role

```
role-selection.html
-> browser event / initial load
-> fetch or authFetch
-> Express mounted route
-> authentication + role middleware when protected
-> controller SQL/business logic
-> JSON response
-> DOM update, toast, or redirect
```

F. User flow

The page validates immediately useful browser input, disables relevant buttons during asynchronous work, then relies on backend validation for ownership, stock, roles, balances, and database integrity. It handles non-success responses by showing the controller's message instead of assuming success.

Frontend Page: search-history.html

A. Purpose

Lists recorded product searches, supports repeat search, individual deletion, and clearing.

B. Access

Customer

C. UI and design

Uses the shared seller.css design: slate page background, dark collapsible role sidebar where appropriate, white panels/cards, blue primary buttons, compact fields, tables, tags, toasts, loading and empty states. Authentication pages use the shared centered auth-card variation.

D. Important JavaScript

- fullDate
- escapeHtml
- drawHistory
- loadHistory
- deleteEntry
- load/draw history
- open products.html with keyword
- DELETE one or all records

E. Backend communication

- /api/customer/search-history
- /api/customer/search-history/

```
search-history.html
-> browser event / initial load
-> fetch or authFetch
-> Express mounted route
-> authentication + role middleware when protected
-> controller SQL/business logic
-> JSON response
-> DOM update, toast, or redirect
```

F. User flow

The page validates immediately useful browser input, disables relevant buttons during asynchronous work, then relies on backend validation for ownership, stock, roles, balances, and database integrity. It handles non-success responses by showing the controller's message instead of assuming success.

Frontend Page: seller-orders.html

A. Purpose

Lists orders containing the seller's products and permits safe status changes.

B. Access

Seller

C. UI and design

Uses the shared seller.css design: slate page background, dark collapsible role sidebar where appropriate, white panels/cards, blue primary buttons, compact fields, tables, tags, toasts, loading and empty states. Authentication pages use the shared centered auth-card variation.

D. Important JavaScript

- loadOrders
- moveOrder
- filter orders
- PATCH status
- block multi-seller action in UI and backend

E. Backend communication

- /api/seller/orders/
- /api/seller/orders?status=

```
seller-orders.html
-> browser event / initial load
-> fetch or authFetch
-> Express mounted route
-> authentication + role middleware when protected
-> controller SQL/business logic
-> JSON response
-> DOM update, toast, or redirect
```

F. User flow

The page validates immediately useful browser input, disables relevant buttons during asynchronous work, then relies on backend validation for ownership, stock, roles, balances, and database integrity. It handles non-success responses by showing the controller's message instead of assuming success.

Frontend Page: seller-product-form.html

A. Purpose

Creates or edits a product with metadata and uploaded images.

B. Access

Seller

C. UI and design

Uses the shared seller.css design: slate page background, dark collapsible role sidebar where appropriate, white panels/cards, blue primary buttons, compact fields, tables, tags, toasts, loading and empty states. Authentication pages use the shared centered auth-card variation.

D. Important JavaScript

- drawExisting
- droplImage
- loadMeta
- loadExisting
- start

- loadMeta
- loadExisting for edit
- build FormData
- POST or PUT product

E. Backend communication

- /api/seller/meta
- /api/seller/products
- /api/seller/products/

```
seller-product-form.html
-> browser event / initial load
-> fetch or authFetch
-> Express mounted route
-> authentication + role middleware when protected
-> controller SQL/business logic
-> JSON response
-> DOM update, toast, or redirect
```

F. User flow

The page validates immediately useful browser input, disables relevant buttons during asynchronous work, then relies on backend validation for ownership, stock, roles, balances, and database integrity. It handles non-success responses by showing the controller's message instead of assuming success.

Frontend Page: seller-product-view.html

A. Purpose

Seller-owned product detail, images, specifications, and reviews.

B. Access

Seller

C. UI and design

Uses the shared seller.css design: slate page background, dark collapsible role sidebar where appropriate, white panels/cards, blue primary buttons, compact fields, tables, tags, toasts, loading and empty states. Authentication pages use the shared centered auth-card variation.

D. Important JavaScript

- showImage
- step
- goTo
- stars
- loadProduct
- load product by id
- image gallery navigation
- render reviews

E. Backend communication

- /api/seller/products/

```
seller-product-view.html
-> browser event / initial load
-> fetch or authFetch
-> Express mounted route
-> authentication + role middleware when protected
-> controller SQL/business logic
-> JSON response
-> DOM update, toast, or redirect
```

F. User flow

The page validates immediately useful browser input, disables relevant buttons during asynchronous work, then relies on backend validation for ownership, stock, roles, balances, and database integrity. It handles non-success responses by showing the controller's message instead of assuming success.

Frontend Page: seller-products.html

A. Purpose

Tabbed list of seller-owned products and product mode controls.

B. Access

Seller

C. UI and design

Uses the shared seller.css design: slate page background, dark collapsible role sidebar where appropriate, white panels/cards, blue primary buttons, compact fields, tables, tags, toasts, loading and empty states. Authentication pages use the shared centered auth-card variation.

D. Important JavaScript

- loadProducts
- removeProduct
- setMode
- GET products by tab
- delete product
- PATCH Active/Inactive mode

E. Backend communication

- /api/seller/products/
- /api/seller/products?tab=

```
seller-products.html
-> browser event / initial load
-> fetch or authFetch
-> Express mounted route
-> authentication + role middleware when protected
-> controller SQL/business logic
-> JSON response
-> DOM update, toast, or redirect
```

F. User flow

The page validates immediately useful browser input, disables relevant buttons during asynchronous work, then relies on backend validation for ownership, stock, roles, balances, and database integrity. It handles non-success responses by showing the controller's message instead of assuming success.

Frontend Page: seller.html

A. Purpose

Dashboard statistics, recent fulfillment work, approval queue, and limited quick actions.

B. Access

Seller

C. UI and design

Uses the shared seller.css design: slate page background, dark collapsible role sidebar where appropriate, white panels/cards, blue primary buttons, compact fields, tables, tags, toasts, loading and empty states. Authentication pages use the shared centered auth-card variation.

D. Important JavaScript

- loadDashboard
- moveOrder
- cancelOrder
- removeProduct
- loadDashboard
- move/cancel eligible single-seller orders
- delete product where allowed
- auto-refresh

E. Backend communication

- /api/seller/dashboard
- /api/seller/orders/
- /api/seller/products/

```
seller.html
-> browser event / initial load
-> fetch or authFetch
-> Express mounted route
-> authentication + role middleware when protected
-> controller SQL/business logic
-> JSON response
-> DOM update, toast, or redirect
```

F. User flow

The page validates immediately useful browser input, disables relevant buttons during asynchronous work, then relies on backend validation for ownership, stock, roles, balances, and database integrity. It handles non-success responses by showing the controller's message instead of assuming success.

Frontend Page: wallet.html

A. Purpose

Read-only money balance, Reward Points, last update, and transaction history.

B. Access

Customer

C. UI and design

Uses the shared seller.css design: slate page background, dark collapsible role sidebar where appropriate, white panels/cards, blue primary buttons, compact fields, tables, tags, toasts, loading and empty states. Authentication pages use the shared centered auth-card variation.

D. Important JavaScript

- fullDate
- escapeHtml
- drawWallet
- loadWallet
- GET /api/customer/wallet
- draw balance cards
- link transactions to owned orders

E. Backend communication

- /api/customer/wallet

```
wallet.html
-> browser event / initial load
-> fetch or authFetch
-> Express mounted route
-> authentication + role middleware when protected
-> controller SQL/business logic
-> JSON response
-> DOM update, toast, or redirect
```

F. User flow

The page validates immediately useful browser input, disables relevant buttons during asynchronous work, then relies on backend validation for ownership, stock, roles, balances, and database integrity. It handles non-success responses by showing the controller's message instead of assuming success.

Frontend Page: wishlist.html

A. Purpose

Displays saved products and moves/adds them to cart or removes them.

B. Access

Customer

C. UI and design

Uses the shared seller.css design: slate page background, dark collapsible role sidebar where appropriate, white panels/cards, blue primary buttons, compact fields, tables, tags, toasts, loading and empty states. Authentication pages use the shared centered auth-card variation.

D. Important JavaScript

- escapeHtml
- stars
- drawWishlist
- loadWishlist
- addToCart
- removeWish
- loadWishlist/drawWishlist
- POST cart
- PATCH wishlist removal

E. Backend communication

- /api/customer/cart
- /api/customer/wishlist
- /api/customer/wishlist/

```
wishlist.html
-> browser event / initial load
-> fetch or authFetch
-> Express mounted route
-> authentication + role middleware when protected
-> controller SQL/business logic
-> JSON response
-> DOM update, toast, or redirect
```

F. User flow

The page validates immediately useful browser input, disables relevant buttons during asynchronous work, then relies on backend validation for ownership, stock, roles, balances, and database integrity. It handles non-success responses by showing the controller's message instead of assuming success.

Part 4 - Backend Code Files

Controller functions receive Express req/res objects. Route parameters are in req.params, query-string filters in req.query, JSON/form values in req.body, and the verified identity in req.user.userId. mysql2 returns [rows, fields] for SELECT and a result object with insertId/affectedRows for writes.

Backend File: controllers/authController.js

Purpose

Authentication, registration, role discovery, and role selection.

Imports

- 'bcrypt'
- 'jsonwebtoken'
- '../db/connection'

Functions

getRoles

Queries CUSTOMER, SELLER, and ADMIN for one UserID and returns only existing memberships.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

redirectForRole

Maps a verified role name to its existing dashboard page.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

login

Finds USER by email, verifies bcrypt hash, discovers roles, signs one-day UserID JWT, updates LastLogin, and chooses direct or role-selection redirect.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

register_seller

Transactionally verifies/reuses an existing USER or creates one, prevents duplicate SELLER membership, inserts seller profile, commits, and signs a token.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

register_customer

Same reuse pattern for CUSTOMER; existing identity fields are not overwritten.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

getMyRoles

Returns database-derived roles for the JWT UserID.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

selectRole

Rejects unknown/unassigned roles and returns a verified dashboard URL.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

Backend File: controllers/adminController.js

Purpose

Admin identity, protected admin creation, customer Reward Point credit, and product approval.

Imports

- './db/connection'
- 'bcrypt'

Functions

registerAdmin

Transactionally adds ADMIN to an existing USER or creates a new USER+ADMIN; route is admin-protected.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

getCustomers

Joins CUSTOMER, USER, and optional WALLET for the Reward Point selector.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

addRewardPoints

Locks customer/wallet, creates missing wallet or increments points, records Reward Credit, and commits.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

getMe

Returns joined USER and ADMIN identity.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

getProducts

Lists products by status with seller/category/brand and status counts.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

updateProductStatus

Allows only Pending, Active, or Rejected changes.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

Backend File: controllers/customerAccountController.js

Purpose

Customer profile endpoint.

Imports

- './db/connection'

Functions

getMe

Joins CUSTOMER to USER and returns the authenticated customer's profile.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

Backend File: controllers/customerCatalogController.js

Purpose

Storefront, product detail, search history, reviews, and helpful votes.

Imports

- './db/connection'

Functions

getProducts

Builds parameterized search/category/brand/price filters and safe sort choices; returns active products and metadata; records non-empty searches.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

getProductById

Loads one active product, seller/category/brand data, wishlist state, delivered-purchase eligibility, images, and reviews.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

getSearchHistory

Returns only the authenticated customer's searches.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

deleteSearchHistory

Deletes one customer-owned search row.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

clearSearchHistory

Deletes all searches for the customer.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

createReview

Validates rating/text, verifies a Delivered order contains the product, inserts review, and recalculates product average.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

markReviewHelpful

Atomically increments HelpfulVotes for an active product review; no one-vote-per-customer table exists.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

Backend File: controllers/customerCartController.js

Purpose

Customer-owned cart and totals.

Imports

- './db/connection'

Functions

getCart

Joins cart to product/image data and calculates subtotal, discount, total, and availability flags.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

addToCart

Validates positive quantity, locks/reads active product stock, then inserts or increments the composite-key cart row without exceeding stock.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

updateCartItem

Validates ownership, quantity, product status, and stock before update.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

removeCartItem

Deletes only by CustomerID and ProductID.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

Backend File: controllers/customerWishlistController.js

Purpose

Customer wishlist.

Imports

- './db/connection'

Functions

getWishlist

Returns saved product cards including current availability.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

updateWishlist

Validates product, INSERT IGNORE adds without duplicates, or DELETE removes based on requested saved state.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

Backend File: controllers/customerAddressController.js

Purpose

Address ownership, validation, defaults, and historical copy-on-write.

Imports

- './db/connection'

Functions

addressFromBody

Normalizes the eight address fields.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

addressError

Checks required receiver, phone, street, city, and district values.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

getAddresses

Lists only addresses owned by the customer.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

createAddress

Inserts an address and makes the first address default.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

updateAddress

Locks the owned address; if referenced by an order, creates a new row so history is unchanged; otherwise updates in place.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

deleteAddress

Deletes only an owned, non-default/unreferenced address according to controller checks.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

setDefaultAddress

Transactionally clears the old default and marks the selected owned row.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

Backend File: controllers/customerOrderController.js

Purpose

Checkout options, order history/details, and transactional checkout.

Imports

- '../db/connection'

Functions

getCheckoutOptions

Returns active payment methods, delivery charge, and the customer's wallet money/points.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

getOrders

Validates filter/sort and returns customer-owned orders.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

getOrderDetails

Validates OrderID, enforces ownership, and returns order, item snapshots, address, method, and transactions.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

placeOrder

Validates and locks address/method/cart/product/wallet data; recalculates totals; handles COD/Card/bKash/Wallet/Reward Points; creates order/items; decrements stock; records payment; clears cart; commits or rolls back.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

Backend File: controllers/customerWalletController.js

Purpose

Read-only customer wallet.

Imports

- './db/connection'

Functions

getWallet

Returns owned WALLET and its TRANSACTION history; hides OrderID unless the joined order belongs to the customer.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

Backend File: controllers/sellerDashboardController.js

Purpose

Seller profile and dashboard aggregates.

Imports

- '../db/connection'

Functions

countRows

Small helper that returns a COUNT query's total.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

getMe

Joins SELLER and USER for seller identity.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

getDashboard

Counts product/order states, sums delivered seller item subtotals, detects multi-seller orders, and returns recent work.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

Backend File: controllers/sellerProductController.js

Purpose

Seller-owned product lifecycle and image persistence.

Imports

- './db/connection'

Functions

getProducts

Lists only the authenticated seller's products by tab.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

getProductById

Returns one seller-owned product, images, category/brand, and reviews.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

getMeta

Returns category and brand choices.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

filesToPaths

Maps uploaded multer file records to public URL paths.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

toArray

Normalizes a single form value or array into an array.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

writeImages

Writes ordered PRODUCT_IMAGE rows for supplied paths.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

createProduct

Validates fields, inserts seller-owned Pending product, writes images, commits.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

updateProduct

Locks seller-owned product, updates allowed fields, optionally replaces images, commits.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

deleteProduct

Prevents deletion when order history references product; otherwise removes dependent current rows and product transactionally.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

updateProductMode

Allows seller to switch Active/Inactive with stock validation.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

Backend File: controllers/sellerOrderController.js

Purpose

Seller order listing and safe status transitions.

Imports

- './db/connection'

Functions

getOrders

Lists order items for this seller and marks mixed-seller orders.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

updateOrderStatus

Locks order, verifies every item belongs to the seller, checks state transition, updates COD delivery payment, restores stock on cancellation, and refunds Reward Points or wallet money to the correct source.

Called through its matching Express route or by another helper. Success sends JSON; validation/ownership failures return a 4xx response; unexpected errors are logged and return a generic 500 message.

Backend File: controllers/customerController.js

Purpose

Customer controller facade.

Imports

This facade only combines exports from dedicated feature controllers.

Functions

Uses object spread with require() to preserve the route-facing controller API while implementation remains separated by feature.

Backend File: controllers/sellerController.js

Purpose

Seller controller facade.

Imports

This facade only combines exports from dedicated feature controllers.

Functions

Uses object spread with require() to preserve the route-facing controller API while implementation remains separated by feature.

Backend File: routes/authRoutes.js

Purpose

Public login/customer/seller registration plus JWT-protected role listing and selection.

Important functions/behavior

This file is primarily declarative configuration or route mapping.

Interaction

It connects to the adjacent layers using require/module.exports. Errors are either passed through middleware responses or caught by controller try/catch blocks.

Backend File: routes/customerRoutes.js

Purpose

Twenty-four customer endpoints; every one uses `authMiddleware` and `customerMiddleware`.

Important functions/behavior

This file is primarily declarative configuration or route mapping.

Interaction

It connects to the adjacent layers using `require/module.exports`. Errors are either passed through middleware responses or caught by controller `try/catch` blocks.

Backend File: routes/sellerRoutes.js

Purpose

Applies auth+seller middleware once; product writes additionally use upload middleware.

Important functions/behavior

This file is primarily declarative configuration or route mapping.

Interaction

It connects to the adjacent layers using require/module.exports. Errors are either passed through middleware responses or caught by controller try/catch blocks.

Backend File: routes/adminRoutes.js

Purpose

Applies auth+admin middleware once; exposes admin registration, rewards, and product approval.

Important functions/behavior

This file is primarily declarative configuration or route mapping.

Interaction

It connects to the adjacent layers using require/module.exports. Errors are either passed through middleware responses or caught by controller try/catch blocks.

Backend File: middleware/authMiddleware.js

Purpose

Reads Bearer token, verifies JWT_SECRET, and sets req.user.userId; otherwise 401.

Important functions/behavior

- authMiddleware

Interaction

It connects to the adjacent layers using require/module.exports. Errors are either passed through middleware responses or caught by controller try/catch blocks.

Backend File: middleware/customerMiddleware.js

Purpose

Queries CUSTOMER for JWT UserID; otherwise 403.

Important functions/behavior

- customerMiddleware

Interaction

It connects to the adjacent layers using require/module.exports. Errors are either passed through middleware responses or caught by controller try/catch blocks.

Backend File: middleware/sellerMiddleware.js

Purpose

Queries SELLER for JWT UserID; otherwise 403.

Important functions/behavior

- sellerMiddleware

Interaction

It connects to the adjacent layers using require/module.exports. Errors are either passed through middleware responses or caught by controller try/catch blocks.

Backend File: middleware/adminMiddleware.js

Purpose

Queries ADMIN, returns 403 if absent, and updates LastActionTime.

Important functions/behavior

- adminMiddleware

Interaction

It connects to the adjacent layers using require/module.exports. Errors are either passed through middleware responses or caught by controller try/catch blocks.

Backend File: middleware/uploadMiddleware.js

Purpose

Multer disk storage, image MIME/extension filter, five-file and size limits, cleanup on failure.

Important functions/behavior

- fileFilter
- uploadProductImages

Interaction

It connects to the adjacent layers using require/module.exports. Errors are either passed through middleware responses or caught by controller try/catch blocks.

Backend File: db/connection.js

Purpose

Creates mysql2/promise pool from environment values; connections are reused.

Important functions/behavior

This file is primarily declarative configuration or route mapping.

Interaction

It connects to the adjacent layers using require/module.exports. Errors are either passed through middleware responses or caught by controller try/catch blocks.

Backend File: server.js

Purpose

Loads environment, JSON parser, static public directory, four API prefixes, and PORT listener.

Important functions/behavior

This file is primarily declarative configuration or route mapping.

Interaction

It connects to the adjacent layers using `require/module.exports`. Errors are either passed through middleware responses or caught by controller `try/catch` blocks.

Backend File: public/panel-common.js

Purpose

Shared token/authFetch, sidebar, identity boxes, formatting, toast, logout, and polling.

Important functions/behavior

- authFetch
- setupSidebar
- loadSellerBox
- loadAdminBox
- initials
- money
- shortDate
- toast
- autoRefresh
- loadCustomerBox

Interaction

It connects to the adjacent layers using require/module.exports. Errors are either passed through middleware responses or caught by controller try/catch blocks.

Backend File: scripts/seedData.js

Purpose

Development-only destructive reseed: clears tables, inserts explicit related data, computes order snapshots/ratings, wallets, transactions, carts, wishlists, and history.

Important functions/behavior

- `escapeXml`
- `writePlaceholder`
- `trim`
- `buildImages`
- `bulk`
- `insertMany`
- `findProduct`
- `run`

Interaction

It connects to the adjacent layers using `require/module.exports`. Errors are either passed through middleware responses or caught by controller `try/catch` blocks.

Part 5 - Authentication

Registration

```
Browser registration form
-> public POST endpoint
-> validate required fields and password length
-> BEGIN transaction
-> SELECT USER by email FOR UPDATE
-> existing email: bcrypt.compare submitted password
-> new email: bcrypt.hash then INSERT USER
-> reject duplicate requested subtype
-> INSERT CUSTOMER or SELLER
-> COMMIT
-> sign JWT and return redirect
```

JWT

A JWT has a Base64URL header, payload, and cryptographic signature. This project stores only `userId` in the payload and sets a one-day expiry. The signature is made with `JWT_SECRET`. `authMiddleware` verifies signature and expiry before trusting `userId`; the secret is never sent to the browser or printed here.

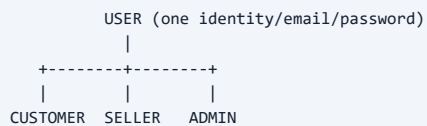
Login

login queries `USER` by email with a placeholder, compares `bcrypt` hash, discovers all `role-table` memberships, updates `LastLogin`, and signs the JWT. One role redirects directly; multiple roles redirect to `role-selection.html`.

Password properties

`bcrypt` hashes are salted and deliberately expensive. A database leak does not reveal the original password directly. Existing-account role registration verifies the current hash and does not overwrite `name`, `phone`, or `password`.

Part 6 - Multiple-Role System



Example:

```
USER.UserID      = 10
CUSTOMER.UserID  = 10
SELLER.UserID    = 10
```

The subtype primary keys are independent foreign keys to USER, so one UserID may exist in any combination of role tables. Email remains unique only once in USER.

Customer + seller login example

1. Login verifies USER 10 password.
2. getRoles finds CUSTOMER 10 and SELLER 10.
3. Response contains roles [customer, seller] and role-selection redirect.
4. Page calls GET /api/auth/roles using JWT.
5. User clicks Seller.
6. POST /api/auth/select-role sends seller.
7. Backend queries memberships and returns /seller.html only if SELLER 10 exists.
8. Every seller API again passes sellerMiddleware.

The role-selection page is convenience, not the security boundary. A modified browser cannot gain admin access because adminMiddleware independently queries ADMIN.

Part 7 - Database Architecture

USER -> CUSTOMER -> ADDRESS, SEARCH_HISTORY, REVIEW, CART_ITEM, WISHLIST_ITEM, WALLET, ORDER
USER -> SELLER -> PRODUCT -> PRODUCT_IMAGE, REVIEW, CART_ITEM, WISHLIST_ITEM, ORDER_ITEM
USER -> ADMIN
CATEGORY -> PRODUCT; CATEGORY -> child CATEGORY
BRAND -> PRODUCT
ORDER -> ORDER_ITEM
ORDER -> TRANSACTION; WALLET -> TRANSACTION
PAYMENT_METHOD -> ORDER

Constraint vocabulary

Construct	Meaning and project example
PRIMARY KEY	Uniquely identifies a row: ProductID; composite (CustomerID, ProductID) in cart/wishlist.
FOREIGN KEY	Requires referenced parent: PRODUCT.SellerID -> SELLER.UserID.
UNIQUE	USER.Email and WALLET.CustomerID cannot repeat.
NOT NULL	Required data such as ProductName and UnitPrice.
DEFAULT	Automatic values such as Pending, zero points, or current timestamp.
CHECK	Rejects invalid ranges such as negative stock or RewardPoints.
AUTO_INCREMENT	MySQL generates identifiers for USER, PRODUCT, ORDER, etc.
CASCADE	Deleting parent removes dependent current data such as cart rows.
RESTRICT	Protects historical/financial references from deletion.
SET NULL	Keeps child/history while removing optional category, brand, or transaction order link.

Database Table: `USER`

Base identity and credentials.

Columns and constraints

UserID	INT	AUTO_INCREMENT	PRIMARY KEY
ProfilePicture	VARCHAR(500)		NULL
PasswordHash	VARCHAR(255)		NOT NULL
Phone	VARCHAR(20)		NULL
Email	VARCHAR(100)		NOT NULL UNIQUE
FullName	VARCHAR(100)		NOT NULL
LastLogin	DATETIME		NULL
RegistrationDate	DATETIME		NOT NULL DEFAULT CURRENT_TIMESTAMP

Relationships

No outgoing foreign key.

Why it exists

Base identity and credentials.

Database Table: CUSTOMER

Customer subtype.

Columns and constraints

UserID	INT	PRIMARY KEY
Gender	VARCHAR(15)	NULL
DateOfBirth	DATE	NULL
	ON DELETE	CASCADE

Relationships

- FOREIGN KEY (UserID) REFERENCES `USER` (UserID)

Why it exists

Customer subtype.

Database Table: SELLER

Seller/store subtype.

Columns and constraints

UserID	INT	PRIMARY KEY
StoreName	VARCHAR(100)	NOT NULL
BusinessPhone	VARCHAR(20)	NULL
BusinessEmail	VARCHAR(100)	NULL
TradeLicenseNumber	VARCHAR(100)	NULL
VerificationStatus	VARCHAR(20)	NOT NULL DEFAULT 'Pending'
JoinedDate	DATETIME	NOT NULL DEFAULT CURRENT_TIMESTAMP
AverageRating	DECIMAL(3,2)	NOT NULL DEFAULT 0
ON DELETE CASCADE		

Relationships

- FOREIGN KEY (UserID) REFERENCES `USER` (UserID)

Why it exists

Seller/store subtype.

Database Table: ADMIN

Administrator subtype.

Columns and constraints

```
UserID INT PRIMARY KEY
AccessLevel VARCHAR(20) NOT NULL DEFAULT 'Standard'
LastActionTime DATETIME NULL
ON DELETE CASCADE
```

Relationships

- FOREIGN KEY (UserID) REFERENCES `USER` (UserID)

Why it exists

Administrator subtype.

Database Table: SEARCH_HISTORY

Customer product-search records.

Columns and constraints

SearchID	INT	AUTO_INCREMENT	PRIMARY KEY
SearchKeyword	VARCHAR(200)	NOT NULL	
SearchTime	DATETIME	NOT NULL	DEFAULT CURRENT_TIMESTAMP
TotalResult	INT	NOT NULL	DEFAULT 0
CustomerID	INT	NOT NULL	
ON DELETE CASCADE			

Relationships

- FOREIGN KEY (CustomerID) REFERENCES CUSTOMER (UserID)

Why it exists

Customer product-search records.

Database Table: ADDRESS

Customer delivery addresses; order references preserve history.

Columns and constraints

```
AddressID INT AUTO_INCREMENT PRIMARY KEY
AddressLabel VARCHAR(100) NULL
ReceiverName VARCHAR(100) NOT NULL
PhoneNumber VARCHAR(20) NOT NULL
StreetAddress VARCHAR(255) NOT NULL
City VARCHAR(50) NOT NULL
District VARCHAR(50) NOT NULL
PostalCode VARCHAR(50) NULL
Country VARCHAR(50) NULL
IsDefault TINYINT NOT NULL DEFAULT 0
UserID INT NOT NULL
ON DELETE CASCADE
```

Relationships

- FOREIGN KEY (UserID) REFERENCES CUSTOMER (UserID)

Why it exists

Customer delivery addresses; order references preserve history.

Database Table: CATEGORY

Hierarchical product categories.

Columns and constraints

CategoryID	INT	AUTO_INCREMENT	PRIMARY KEY
CategoryName	VARCHAR(100)	NOT NULL	
Description	VARCHAR(300)	NULL	
ParentID	INT	NULL	
		ON DELETE SET NULL	

Relationships

- FOREIGN KEY (ParentID) REFERENCES CATEGORY (CategoryID)

Why it exists

Hierarchical product categories.

Database Table: BRAND

Product brand metadata.

Columns and constraints

BrandID	INT	AUTO_INCREMENT	PRIMARY KEY
BrandName	VARCHAR(100)	NOT NULL	
Country	VARCHAR(50)	NULL	
Website	VARCHAR(200)	NULL	
LogoURL	VARCHAR(500)	NULL	

Relationships

No outgoing foreign key.

Why it exists

Product brand metadata.

Database Table: PRODUCT

Seller-owned catalog and inventory.

Columns and constraints

```
ProductID INT AUTO_INCREMENT PRIMARY KEY
ProductName VARCHAR(100) NOT NULL
Description TEXT NULL
UnitPrice DECIMAL(10,2) NOT NULL
Discount DECIMAL(5,2) NOT NULL DEFAULT 0
AverageRating DECIMAL(3,2) NOT NULL DEFAULT 0
StockQuantity INT NOT NULL DEFAULT 0
Weight DECIMAL(8,2) NULL
Warranty VARCHAR(100) NULL
Status VARCHAR(50) NOT NULL DEFAULT 'Pending'
CreatedAt DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
UpdatedAt DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
CategoryID INT NULL
BrandID INT NULL
SellerID INT NOT NULL
ON DELETE SET NULL
ON DELETE SET NULL
ON DELETE RESTRICT
```

Relationships

- FOREIGN KEY (CategoryID) REFERENCES CATEGORY (CategoryID)
- FOREIGN KEY (BrandID) REFERENCES BRAND (BrandID)
- FOREIGN KEY (SellerID) REFERENCES SELLER (UserID)

Why it exists

Seller-owned catalog and inventory.

Database Table: PRODUCT_IMAGE

Ordered images with composite key.

Columns and constraints

ImageNumber INT NOT NULL
ImageURL VARCHAR(500) NOT NULL
Caption VARCHAR(200) NULL
DisplayOrder INT NOT NULL DEFAULT 1
IsPrimary TINYINT NOT NULL DEFAULT 0
ProductID INT NOT NULL
ON DELETE CASCADE

Relationships

- FOREIGN KEY (ProductID) REFERENCES PRODUCT (ProductID)

Why it exists

Ordered images with composite key.

Database Table: REVIEW

Customer ratings/text/helpful count.

Columns and constraints

```
ReviewID INT AUTO_INCREMENT PRIMARY KEY
HelpfulVotes INT NOT NULL DEFAULT 0
ReviewText VARCHAR(1000) NULL
Rating INT NOT NULL
ReviewDate DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
CustomerID INT NOT NULL
ProductID INT NOT NULL
ON DELETE CASCADE
ON DELETE CASCADE
```

Relationships

- FOREIGN KEY (CustomerID) REFERENCES CUSTOMER (UserID)
- FOREIGN KEY (ProductID) REFERENCES PRODUCT (ProductID)

Why it exists

Customer ratings/text/helpful count.

Database Table: WISHLIST_ITEM

Customer-product saved relation.

Columns and constraints

SavedAt	DATETIME	NOT NULL	DEFAULT CURRENT_TIMESTAMP
CustomerID	INT	NOT NULL	
ProductID	INT	NOT NULL	
	ON DELETE	CASCADE	
	ON DELETE	CASCADE	

Relationships

- FOREIGN KEY (CustomerID) REFERENCES CUSTOMER (UserID)
- FOREIGN KEY (ProductID) REFERENCES PRODUCT (ProductID)

Why it exists

Customer-product saved relation.

Database Table: CART_ITEM

Customer-product quantity relation.

Columns and constraints

AddedAt	DATETIME	NOT NULL	DEFAULT CURRENT_TIMESTAMP
Quantity	INT	NOT NULL	DEFAULT 1
CustomerID	INT	NOT NULL	
ProductID	INT	NOT NULL	
ON DELETE CASCADE			
ON DELETE CASCADE			

Relationships

- FOREIGN KEY (CustomerID) REFERENCES CUSTOMER (UserID)
- FOREIGN KEY (ProductID) REFERENCES PRODUCT (ProductID)

Why it exists

Customer-product quantity relation.

Database Table: WALLET

One customer wallet with money and Reward Points.

Columns and constraints

```
WalletID INT AUTO_INCREMENT PRIMARY KEY
CurrentBalance DECIMAL(10,2) NOT NULL DEFAULT 0
RewardPoints INT NOT NULL DEFAULT 0
LastUpdated DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
CustomerID INT NOT NULL UNIQUE
ON DELETE CASCADE
```

Relationships

- FOREIGN KEY (CustomerID) REFERENCES CUSTOMER (UserID)

Why it exists

One customer wallet with money and Reward Points.

Database Table: PAYMENT_METHOD

Active checkout methods.

Columns and constraints

PaymentMethodID	INT	AUTO_INCREMENT	PRIMARY KEY
MethodName	VARCHAR(30)	NOT NULL	
Description	VARCHAR(300)	NULL	
IsActive	TINYINT	NOT NULL	DEFAULT 1

Relationships

No outgoing foreign key.

Why it exists

Active checkout methods.

Database Table: `ORDER`

Order header, address, totals, statuses, method.

Columns and constraints

```
OrderID INT AUTO_INCREMENT PRIMARY KEY
OrderDate DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
TotalAmount DECIMAL(10,2) NOT NULL
DiscountAmount DECIMAL(10,2) NOT NULL DEFAULT 0
DeliveryCharge DECIMAL(10,2) NOT NULL DEFAULT 0
OrderStatus VARCHAR(50) NOT NULL DEFAULT 'Pending'
PaymentStatus VARCHAR(50) NOT NULL DEFAULT 'Due'
CreatedAt DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
CustomerID INT NOT NULL
AddressID INT NULL
PaymentMethodID INT NULL
ON DELETE RESTRICT
ON DELETE RESTRICT
ON DELETE RESTRICT
```

Relationships

- FOREIGN KEY (CustomerID) REFERENCES CUSTOMER (UserID)
- FOREIGN KEY (AddressID) REFERENCES ADDRESS (AddressID)
- FOREIGN KEY (PaymentMethodID) REFERENCES PAYMENT_METHOD (PaymentMethodID)

Why it exists

Order header, address, totals, statuses, method.

Database Table: `TRANSACTION`

Wallet/reward payment, credit, and refund history.

Columns and constraints

```
TransactionID INT AUTO_INCREMENT PRIMARY KEY
Status VARCHAR(20) NOT NULL DEFAULT 'Pending'
TransactionDate DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
TransactionType VARCHAR(30) NOT NULL
Amount DECIMAL(10,2) NOT NULL
WalletID INT NOT NULL
OrderID INT NULL
ON DELETE RESTRICT
ON DELETE SET NULL
```

Relationships

- FOREIGN KEY (WalletID) REFERENCES WALLET (WalletID)
- FOREIGN KEY (OrderID) REFERENCES `ORDER` (OrderID)

Why it exists

Wallet/reward payment, credit, and refund history.

Database Table: ORDER_ITEM

Immutable per-order product price/discount/quantity snapshots.

Columns and constraints

ItemNo	INT	NOT NULL
Quantity	INT	NOT NULL
UnitPrice	DECIMAL(10,2)	NOT NULL
Discount	DECIMAL(5,2)	NOT NULL DEFAULT 0
SubTotal	DECIMAL(10,2)	NOT NULL
OrderID	INT	NOT NULL
ProductID	INT	NOT NULL
ON DELETE CASCADE		
ON DELETE RESTRICT		

Relationships

- FOREIGN KEY (OrderID) REFERENCES `ORDER` (OrderID)
- FOREIGN KEY (ProductID) REFERENCES PRODUCT (ProductID)

Why it exists

Immutable per-order product price/discount/quantity snapshots.

Part 8 - Major Business Flows

The diagrams below summarize actual implementations. Each arrow crosses a concrete page, route, controller, or database boundary.

Business Flow: Customer registration

```
register_customer.html
-> POST auth
-> USER lookup/lock
-> password verify or USER insert
-> CUSTOMER insert
-> commit
-> JWT.
```

Important guarantees

Client values are treated as requests, not authority. Role, ownership, stock, totals, payment balances, and eligible transitions are checked by backend queries. Multi-step writes use one pooled connection and transaction when partial state would be unsafe.

Business Flow: Login

```
login.html
-> USER lookup
-> bcrypt
-> role tables
-> JWT
-> direct dashboard or role selector.
```

Important guarantees

Client values are treated as requests, not authority. Role, ownership, stock, totals, payment balances, and eligible transitions are checked by backend queries. Multi-step writes use one pooled connection and transaction when partial state would be unsafe.

Business Flow: Product browsing/search

```
products.html
-> GET products filters
-> parameterized PRODUCT query
-> active cards; non-empty keyword inserts SEARCH_HISTORY.
```

Important guarantees

Client values are treated as requests, not authority. Role, ownership, stock, totals, payment balances, and eligible transitions are checked by backend queries. Multi-step writes use one pooled connection and transaction when partial state would be unsafe.

Business Flow: Product details/review

```
detail id
-> joined product/seller/category/brand
-> images/reviews
-> Delivered ORDER+ORDER_ITEM required before review insert.
```

Important guarantees

Client values are treated as requests, not authority. Role, ownership, stock, totals, payment balances, and eligible transitions are checked by backend queries. Multi-step writes use one pooled connection and transaction when partial state would be unsafe.

Business Flow: Cart

```
POST/PATCH/DELETE cart
-> customer role
-> product status/stock checks
-> composite CART_ITEM update
-> recalculated totals on GET.
```

Important guarantees

Client values are treated as requests, not authority. Role, ownership, stock, totals, payment balances, and eligible transitions are checked by backend queries. Multi-step writes use one pooled connection and transaction when partial state would be unsafe.

Business Flow: Wishlist

```
PATCH saved state
-> validate active product
-> INSERT IGNORE or customer-owned DELETE.
```

Important guarantees

Client values are treated as requests, not authority. Role, ownership, stock, totals, payment balances, and eligible transitions are checked by backend queries. Multi-step writes use one pooled connection and transaction when partial state would be unsafe.

Business Flow: Address

owned CRUD

- > referenced row uses copy-on-write
- > existing ORDER keeps original address row.

Important guarantees

Client values are treated as requests, not authority. Role, ownership, stock, totals, payment balances, and eligible transitions are checked by backend queries. Multi-step writes use one pooled connection and transaction when partial state would be unsafe.

Business Flow: Checkout

```
validate/lock/recalculate
-> order and item snapshots
-> stock deduction
-> optional payment
-> cart clear
-> commit.
```

Important guarantees

Client values are treated as requests, not authority. Role, ownership, stock, totals, payment balances, and eligible transitions are checked by backend queries. Multi-step writes use one pooled connection and transaction when partial state would be unsafe.

Business Flow: Wallet and rewards

wallet.html

-> owned wallet and transactions; Reward Credit/Payment/Refund use existing TRANSACTION.

Important guarantees

Client values are treated as requests, not authority. Role, ownership, stock, totals, payment balances, and eligible transitions are checked by backend queries. Multi-step writes use one pooled connection and transaction when partial state would be unsafe.

Business Flow: Seller product creation

```
multipart form
-> upload middleware
-> validation
-> PRODUCT Pending
-> PRODUCT_IMAGE rows
-> commit
-> admin approval.
```

Important guarantees

Client values are treated as requests, not authority. Role, ownership, stock, totals, payment balances, and eligible transitions are checked by backend queries. Multi-step writes use one pooled connection and transaction when partial state would be unsafe.

Business Flow: Seller order update

```
seller ownership aggregation
-> reject mixed-seller action
-> legal transition
-> status/COD/stock/refund writes
-> commit.
```

Important guarantees

Client values are treated as requests, not authority. Role, ownership, stock, totals, payment balances, and eligible transitions are checked by backend queries. Multi-step writes use one pooled connection and transaction when partial state would be unsafe.

Business Flow: Admin

admin middleware

-> product approval, protected admin role assignment, customer Reward Point listing/credit.

Important guarantees

Client values are treated as requests, not authority. Role, ownership, stock, totals, payment balances, and eligible transitions are checked by backend queries. Multi-step writes use one pooled connection and transaction when partial state would be unsafe.

Business Flow: Customer cancellation (actual behavior)

No customer cancellation route/page exists. Eligible cancellation is initiated by seller for a single-seller order.

Important guarantees

Client values are treated as requests, not authority. Role, ownership, stock, totals, payment balances, and eligible transitions are checked by backend queries. Multi-step writes use one pooled connection and transaction when partial state would be unsafe.

Part 9 - Reward Point System

One whole Reward Point equals one whole taka. WALLET.RewardPoints is an integer and cannot be negative. Full point payment is offered only for a whole-taka backend total; mixed payment is not implemented.

Admin credit

```
admin.html
-> GET /api/admin/customers
-> select customer and positive integer
-> POST /api/admin/customers/:customerId/reward-points
-> authMiddleware + adminMiddleware
-> lock CUSTOMER and WALLET
-> create missing WALLET or RewardPoints += points
-> INSERT TRANSACTION type Reward Credit
-> COMMIT
```

Full Reward Point payment

```
checkout options returns actual RewardPoints
-> page offers option only when points > 0, enough, total whole
-> POST checkout with useRewardPoints=true
-> backend recalculates total and locks WALLET
-> conditional RewardPoints = RewardPoints - total WHERE RewardPoints >= total
-> INSERT Reward Payment linked to order
-> COMMIT
```

Cancellation

```
seller cancels eligible paid order
-> find successful Reward Payment for OrderID
-> RewardPoints += original Amount
-> INSERT Reward Refund
-> do not change CurrentBalance
-> COMMIT
```

Example: a whole-taka order of 1000 with 1000 points uses exactly 1000 points, leaves zero, marks the order Paid, and records Reward Payment amount 1000.

Part 10 - Checkout and Database Transactions

BEGIN, COMMIT, ROLLBACK

BEGIN starts an isolated unit of work on one pooled connection. COMMIT makes every write permanent. ROLLBACK undoes all writes since BEGIN. Releasing the connection returns it to the pool.

Why FOR UPDATE matters

A normal SELECT reads data but does not reserve it. SELECT ... FOR UPDATE locks matching rows until commit/rollback. Checkout locks owned address, payment method, cart/product rows, and wallet where needed so concurrent requests cannot both act on the same old balance.

Request A	Request B
BEGIN	BEGIN
SELECT WALLET FOR UPDATE (lock)	SELECT WALLET FOR UPDATE (waits)
reads 1000 points	
deducts 1000	
creates order and transaction	
COMMIT (releases lock)	resumes and reads 0
	insufficient -> ROLLBACK

Checkout write order

```
validate address and active payment method
lock/read cart + products
validate active status, positive quantity, sufficient stock
calculate subtotal, discount, delivery, grand total
lock/check wallet if money or rewards
INSERT ORDER
INSERT each ORDER_ITEM snapshot
conditional stock decrement
deduct wallet money OR whole Reward Points
INSERT payment transaction when appropriate
DELETE CART_ITEM
COMMIT
```

A forced error at any point reaches catch, executes rollback, logs the internal error, returns HTTP 500, and leaves order, stock, wallet, points, and cart unchanged.

Part 11 - API Reference

All protected calls require Authorization: Bearer <JWT>. Customer, seller, and admin labels mean both JWT authentication and the matching role-table check. All errors use JSON messages; unexpected errors generally return 500 Server error.

API Group: Authentication

Method	URL	Access	Input	Controller	Result/errors
POST	/api/auth/login	Public	email,password	login	200 token/roles/redirect; 400/401/403/500
POST	/api/auth/register_customer	Public	fullName,email,password,phone,gender,dateOfBirth	register_customer	201 token; 400/401/409/500
POST	/api/auth/register_seller	Public	identity + store fields	register_seller	201 token; 400/401/409/500
GET	/api/auth/roles	JWT	-	getMyRoles	roles or 401/500
POST	/api/auth/select-role	JWT	role	selectRole	redirect or 400/403

API Group: Customer

Method	URL	Access	Input	Controller	Result/errors
GET	/api/customer/me	Customer	-	getMe	customer profile
GET	/api/customer/products	Customer	search/categoryId/brandId/minPrice/maxPrice/sort	getProducts	products/categories/brands
GET	/api/customer/products/:productId	Customer	path id	getProductById	product/images/reviews/eligibility
POST	/api/customer/products/:productId/reviews	Customer	rating,reviewText	createReview	201 or purchase/validation errors
PATCH	/api/customer/reviews/:reviewId/helpful	Customer	-	markReviewHelpful	incremented HelpfulVotes
GET	/api/customer/search-history	Customer	-	getSearchHistory	owned history
DELETE	/api/customer/search-history	Customer	-	clearSearchHistory	cleared
DELETE	/api/customer/search-history/:searchId	Customer	path id	deleteSearchHistory	deleted/not found
GET	/api/customer/cart	Customer	-	getCart	items + summary
POST	/api/customer/cart	Customer	productId,quantity	addToCart	added/stock errors
PATCH	/api/customer/cart/:productId	Customer	quantity	updateCartItem	updated/stock errors
DELETE	/api/customer/cart/:productId	Customer	-	removeCartItem	removed/not found
GET	/api/customer/wishlist	Customer	-	getWishlist	owned items
PATCH	/api/customer/wishlist/:productId	Customer	saved	updateWishlist	saved/removed
GET	/api/customer/addresses	Customer	-	getAddresses	owned addresses
POST	/api/customer/addresses	Customer	address fields	createAddress	201 addressId
PUT	/api/customer/addresses/:addressId	Customer	address fields	updateAddress	updated or copied
DELETE	/api/customer/addresses/:addressId	Customer	-	deleteAddress	deleted/errors
PATCH	/api/customer/addresses/:addressId/default	Customer	-	setDefaultAddress	default set
GET	/api/customer/wallet	Customer	-	getWallet	wallet + transactions
GET	/api/customer/orders	Customer	status,sort	getOrders	owned orders
GET	/api/customer/orders/:orderId	Customer	path id	getOrderDetails	order/items/transactions
GET	/api/customer/checkout/options	Customer	-	getCheckoutOptions	methods, wallet, delivery
POST	/api/customer/checkout	Customer	addressId,paymentMethodId, optional useRewardPoints	placeOrder	201 order or validation/conflict/500

API Group: Seller

Method	URL	Access	Input	Controller	Result/errors
GET	/api/seller/me	Seller	-	getMe	seller profile
GET	/api/seller/dashboard	Seller	-	getDashboard	stats/work queues
GET	/api/seller/meta	Seller	-	getMeta	categories/brands
GET	/api/seller/products	Seller	tab	getProducts	owned products
GET	/api/seller/products/:id	Seller	path id	getProductById	owned product detail
POST	/api/seller/products	Seller + upload	multipart product/images	createProduct	201 product
PUT	/api/seller/products/:id	Seller + upload	multipart fields/images	updateProduct	updated
DELETE	/api/seller/products/:id	Seller	path id	deleteProduct	deleted/history conflict
PATCH	/api/seller/products/:id/mode	Seller	mode,stockQuantity	updateProductMode	Active/Inactive
GET	/api/seller/orders	Seller	status	getOrders	seller order items
PATCH	/api/seller/orders/:orderId/status	Seller	newStatus	updateOrderStatus	updated/mixed/illegal errors

API Group: Admin

Method	URL	Access	Input	Controller	Result/errors
GET	/api/admin/me	Admin	-	getMe	admin profile
POST	/api/admin/register	Admin	email; new user fullName/password/phone	registerAdmin	admin role added
GET	/api/admin/customers	Admin	-	getCustomers	customers + points
POST	/api/admin/customers/:customerId/reward-points	Admin	points	addRewardPoints	additive credit
GET	/api/admin/products	Admin	status	getProducts	products/counts
PATCH	/api/admin/products/:id/status	Admin	newStatus	updateProductStatus	status updated

Part 12 - Important Code Explained

```
const [rows] = await db.query(  
  'SELECT UserID FROM CUSTOMER WHERE UserID = ?',  
  [req.user.userId]  
);
```

- await pauses this async function until the promise resolves without blocking the whole Node process.
- db.query sends SQL through the mysql2 pool.
- ? is a placeholder, not string concatenation.
- [req.user.userId] supplies the placeholder value separately, helping prevent SQL injection.
- mysql2 returns a two-element result; destructuring [rows] keeps the row array.

```
const conn = await db.getConnection();  
await conn.beginTransaction();  
try {  
  // all related reads and writes use conn  
  await conn.commit();  
} catch (err) {  
  await conn.rollback();  
} finally {  
  conn.release();  
}
```

A transaction must use the same physical pooled connection. Using db.query for some writes and conn.query for others would place operations on different connections and break atomicity.

Express request and response

req.body is parsed JSON, req.params contains path placeholders, req.query contains filters, and req.user is assigned only after JWT verification. res.status(n).json(object) selects an HTTP status and serializes JSON.

affectedRows and insertId

INSERT results provide insertId for generated ProductID/OrderID/etc. UPDATE/DELETE affectedRows lets controllers detect missing or concurrently changed records.

Part 13 - Security

Mechanism	Actual implementation
Password hashing	bcrypt hash cost 10 for real registration; compare at login and existing-account role addition.
JWT	Signed one-day token containing userId only.
Authentication	Bearer header required by authMiddleware.
Authorization	CUSTOMER/SELLER/ADMIN membership queried server-side; ownership IDs added to SQL WHERE clauses.
SQL injection defense	User values passed as mysql2 placeholders; sort/status choices are allow-listed before SQL fragments.
Transactions/locks	Checkout, role creation, addresses, product writes, seller order updates, and reward credits use transactions as needed.
Upload validation	Multer limits count/size and accepts image extensions/MIME types.
Negative points	Schema CHECK plus positive admin validation plus conditional checkout UPDATE.
Duplicate users	UNIQUE Email and existing-user reuse after password verification.
Admin assignment	Only existing admins reach POST /api/admin/register.

Current limitations

- JWT is stored in localStorage, so a successful cross-site scripting flaw could read it.
- No CSRF token is used; Bearer tokens are not automatically sent like cookies, reducing classic CSRF risk.
- No rate limiting or account lockout is implemented for login.
- No password reset, email verification, refresh-token, or logout revocation mechanism exists.
- Helpful voting has no per-customer vote record, by approved design.
- Reward Credit transactions do not store which admin performed the credit because TRANSACTION has no AdminID.
- Several frontend renderers escape dynamic text, but not every legacy panel interpolation consistently applies escaping.
- The server listen callback does not report Express 5 listen errors; an occupied port can print a misleading success message.

Part 14 - Error Handling

database/validation problem

- > controller detects condition or catch receives exception
- > transaction path calls ROLLBACK
- > controller sends 400/401/403/404/409/500 JSON
- > authFetch receives response
- > 401 clears token and redirects
- > other errors become toast, inline message, empty/error state
- > buttons are re-enabled where appropriate

Status	Meaning in this project	Example
400	Malformed/invalid request	invalid IDs, role, status, address fields, decimal reward total
401	Not authenticated or wrong credentials	missing/expired token; incorrect existing-user password
403	Authenticated but unauthorized	wrong role, seller does not own order
404	Owned/target resource absent	order/product/address not found
409	Valid request conflicts with current state	stock insufficient, duplicate role, illegal order transition, insufficient points
500	Unexpected server/database failure	catch logs error; transaction rolls back

Part 15 - What Happens When I Click...

When I Click: Login

```
form submit
-> POST login
-> USER/bcrypt/roles/JWT
-> token saved
-> redirect.
```

The frontend improves usability, but the controller repeats security and business validation because browser code can be changed by the user.

When I Click: Add to Cart

```
button
-> POST cart
-> customer auth
-> product/stock validation
-> insert/increment CART_ITEM
-> toast.
```

The frontend improves usability, but the controller repeats security and business validation because browser code can be changed by the user.

When I Click: Checkout

```
button
-> POST checkout
-> all validation/locks/calculation/writes
-> commit
-> order-details redirect.
```

The frontend improves usability, but the controller repeats security and business validation because browser code can be changed by the user.

When I Click: Pay with Reward Points

```
special wallet option
-> backend whole-total and actual-point checks
-> conditional deduction
-> Reward Payment
-> commit.
```

The frontend improves usability, but the controller repeats security and business validation because browser code can be changed by the user.

When I Click: Admin Add Points

```
select customer/points
-> protected POST
-> lock/create/update wallet
-> Reward Credit
-> refreshed current points.
```

The frontend improves usability, but the controller repeats security and business validation because browser code can be changed by the user.

When I Click: Seller Update Order

PATCH status

- > seller ownership and mixed-order checks
- > legal transition
- > COD/stock/refund changes
- > commit.

The frontend improves usability, but the controller repeats security and business validation because browser code can be changed by the user.

When I Click: Write Review

```
submit
-> delivered purchase verification
-> REVIEW insert
-> PRODUCT average recompute.
```

The frontend improves usability, but the controller repeats security and business validation because browser code can be changed by the user.

When I Click: Edit Used Address

PUT

- > detect ORDER reference
- > insert copied edited address
- > old row remains historical.

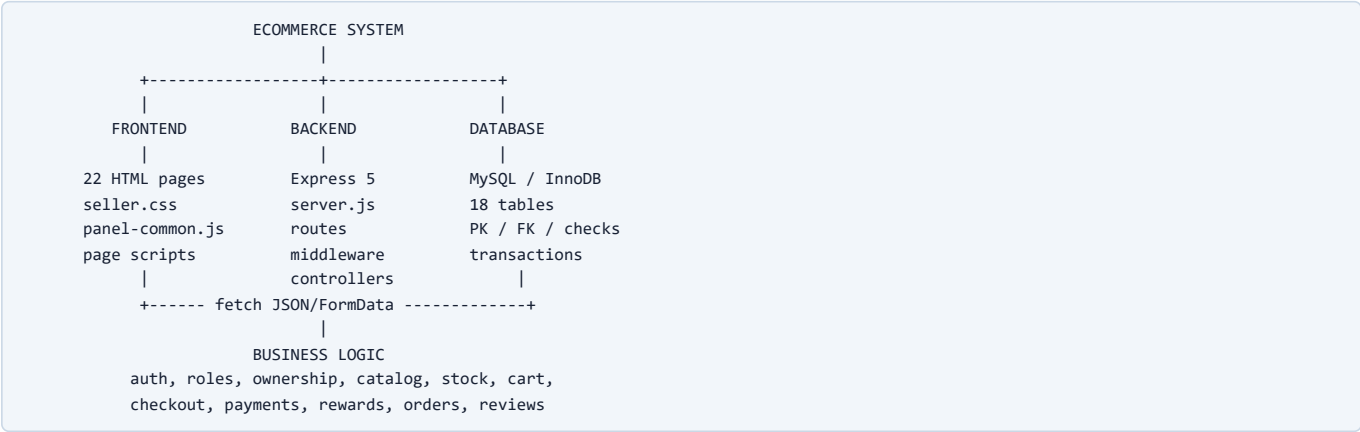
The frontend improves usability, but the controller repeats security and business validation because browser code can be changed by the user.

Part 16 - File-by-File Quick Reference

File	Purpose	Important functions/content
controllers/adminController.js	Admin identity, protected admin creation, customer Reward Point credit, and product approval.	registerAdmin, getCustomers, addRewardPoints, getMe, getProducts, updateProductStatus
controllers/authController.js	Authentication, registration, role discovery, and role selection.	getRoles, redirectForRole, login, register_seller, register_customer, getMyRoles, selectRole
controllers/customerAccountController.js	Customer profile endpoint.	getMe
controllers/customerAddressController.js	Address ownership, validation, defaults, and historical copy-on-write.	addressFromBody, addressError, getAddresses, createAddress, updateAddress, deleteAddress, setDefaultAddress
controllers/customerCartController.js	Customer-owned cart and totals.	getCart, addToCart, updateCartItem, removeCartItem
controllers/customerCatalogController.js	Storefront, product detail, search history, reviews, and helpful votes.	getProducts, getProductById, getSearchHistory, deleteSearchHistory, clearSearchHistory, createReview, markReviewHelpful
controllers/customerController.js	Customer controller facade.	facade exports
controllers/customerOrderController.js	Checkout options, order history/details, and transactional checkout.	getCheckoutOptions, getOrders, getOrderDetails, placeOrder
controllers/customerWalletController.js	Read-only customer wallet.	getWallet
controllers/customerWishlistController.js	Customer wishlist.	getWishlist, updateWishlist
controllers/sellerController.js	Seller controller facade.	facade exports
controllers/sellerDashboardController.js	Seller profile and dashboard aggregates.	countRows, getMe, getDashboard
controllers/sellerOrderController.js	Seller order listing and safe status transitions.	getOrders, updateOrderStatus
controllers/sellerProductController.js	Seller-owned product lifecycle and image persistence.	getProducts, getProductById, getMeta, filesToPaths, toArray, writeImages, createProduct, updateProduct, deleteProduct, updateProductMode
db/connection.js	Project runtime/configuration.	see dedicated section
db/ecommerce_dump.sql	Database definition/legacy SQL.	DDL
middleware/adminMiddleware.js	Request security/upload middleware.	middleware function
middleware/authMiddleware.js	Request security/upload middleware.	middleware function
middleware/customerMiddleware.js	Request security/upload middleware.	middleware function
middleware/sellerMiddleware.js	Request security/upload middleware.	middleware function
middleware/uploadMiddleware.js	Request security/upload middleware.	middleware function
mydb.sql	Database definition/legacy SQL.	DDL
public/addresses.html	CRUD and default selection for customer-owned addresses.	escapeHtml, drawAddresses, loadAddresses, clearForm, editAddress, setDefault, deleteAddress
public/admin.html	Product approval plus customer Reward Point crediting.	loadProducts, setStatus, loadRewardCustomers, showCurrentPoints
public/cart.html	Displays cart totals and allows quantity changes/removal before checkout.	escapeHtml, unavailableMessage, drawCart, loadCart, changeQuantity, removeItem
public/checkout.html	Loads cart/address/payment choices and atomically places an order, including full Reward Point payment.	escapeHtml, drawCheckout, loadCheckout, toggleAddressForm, addAddress, placeOrder
public/customer.html	Displays the customer account profile in the shared customer panel layout.	escapeHtml, loadCustomer
public/login.html	Authenticates an account with email/password; stores JWT and follows the server-selected redirect.	form handlers
public/order-details.html	Owned order header, address, products, totals, payment method, and transaction history.	fullDate, escapeHtml, addressText, drawOrder, loadOrder
public/orders.html	Filtered and sorted order history.	loadOrders, orderDate, escapeHtml
public/panel-common.js	Shared panel browser helpers.	authFetch, sidebar, identity, toast, polling
public/product-details.html	Product gallery, specifications, seller details, reviews, cart and wishlist actions.	escapeHtml, stars, showImage, step, goTo, drawProduct, drawReviews, loadProduct, addToCart, toggleWishlist, submitReview, markHelpful
public/products.html	Searchable/filterable/sortable storefront for active products.	escapeHtml, stars, drawFilters, drawProducts, buildQuery, loadProducts, addToCart, toggleWishlist
public/register_admin.html	Allows an authenticated admin to assign ADMIN to an existing USER or create USER+ADMIN.	form handlers
public/register_customer.html	Creates a new USER+CUSTOMER or adds CUSTOMER to an existing password-verified USER.	form handlers

File	Purpose	Important functions/content
public/register_seller.html	Creates USER+SELLER or adds SELLER to an existing password-verified USER.	form handlers
public/role-selection.html	Lets a multi-role account choose only a role actually assigned by the database.	loadRoles, chooseRole
public/search-history.html	Lists recorded product searches, supports repeat search, individual deletion, and clearing.	fullDate, escapeHtml, drawHistory, loadHistory, deleteEntry
public/seller-orders.html	Lists orders containing the seller's products and permits safe status changes.	loadOrders, moveOrder
public/seller-product-form.html	Creates or edits a product with metadata and uploaded images.	drawExisting, dropImage, loadMeta, loadExisting, start
public/seller-product-view.html	Seller-owned product detail, images, specifications, and reviews.	showImage, step, goTo, stars, loadProduct
public/seller-products.html	Tabbed list of seller-owned products and product mode controls.	loadProducts, removeProduct, setMode
public/seller.css	Shared site styling.	CSS classes
public/seller.html	Dashboard statistics, recent fulfillment work, approval queue, and limited quick actions.	loadDashboard, moveOrder, cancelOrder, removeProduct
public/wallet.html	Read-only money balance, Reward Points, last update, and transaction history.	fullDate, escapeHtml, drawWallet, loadWallet
public/wishlist.html	Displays saved products and moves/adds them to cart or removes them.	escapeHtml, stars, drawWishlist, loadWishlist, addToCart, removeWish
routes/adminRoutes.js	Express route mapping.	router method bindings
routes/authRoutes.js	Express route mapping.	router method bindings
routes/customerRoutes.js	Express route mapping.	router method bindings
routes/sellerRoutes.js	Express route mapping.	router method bindings
scripts/seedData.js	Project runtime/configuration.	see dedicated section
server.js	Project runtime/configuration.	see dedicated section

Part 17 - Final System Map



Expanded map

- Public authentication pages call auth routes directly.
- Protected pages use panel-common.js authFetch and a Bearer JWT.
- Mounted route groups are /api/auth, /api/customer, /api/seller, and /api/admin.
- Middleware establishes identity and database-confirmed role.
- Controllers validate inputs, execute parameterized SQL, and coordinate transactions.
- MySQL foreign keys preserve relationships; snapshots preserve historical order price/address meaning.
- JSON responses drive DOM rendering, messages, loading/empty states, and navigation.

Study advice

To follow a feature, start at its HTML fetch call, locate the matching route, note middleware, open the exported controller function, then match each SQL column to ecommerce_dump.sql. For multi-step writes, trace every statement between beginTransaction and commit/rollback.

Documentation completeness: all current non-dependency project source files, all 22 HTML pages, all controller functions, all route and middleware modules, all 18 authoritative schema tables, and all 46 current API endpoints were analyzed. Secrets in .env and dependency internals under node_modules were intentionally excluded.